

Ponteiros

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br

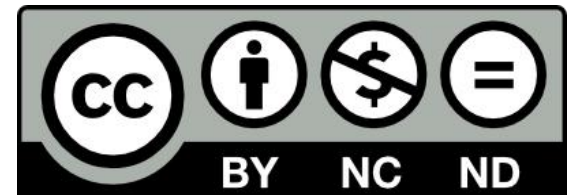


UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO



INSTITUTO DE
COMPUTAÇÃO
UFRJ



Sumário

- Ponteiros
- Exemplos
- Exercícios

- O conceito de **ponteiro** está diretamente ligado à **manipulação da memória do computador**
- Fundamental para a implementação de estruturas de dados muito importantes como **listas, pilhas e árvores**

Conceito de ponteiro

- Mecanismo para **referenciar** um conteúdo indiretamente, através da **posição** em que o **conteúdo** está.
- **Metáfora**: Diferenciar entre o **endereço** de um assento no show e a **pessoa** que está sentada no assento específico.



Maria está sentada no
assento 50 da fila 118



Organização da memória

- A memória do computador é organizada como um **vetor de células endereçadas** consecutivamente que podem ser manipuladas individualmente (ex., char) ou em grupos (ex., short int, int, float, etc.)
- Sempre que usamos o **nome** de uma variável, estamos referenciando simbolicamente o conteúdo que está em um **endereço**, embora não seja necessário conhecer esse endereço.

```
int main(){  
    int x = 1;  
    int y = 10;  
  
    return 0;  
}
```

“Endereço de x é 1000”

“Endereço de y é 1004”

Memória:

[endereço] conteúdo

[1000]	1
[1004]	10
[1008]	
[1009]	
[1010]	

O que são ponteiros?

- É um espaço reservado de memória usado para guardar o endereço de memória de uma outra variável.
- Um ponteiro é uma **variável** como qualquer outra do programa – sua diferença é que ela não armazena um valor inteiro, real, caractere ou booleano. Ela serve para armazenar **endereços de memória** (são valores inteiros sem sinal).

Memória:

[endereço] conteúdo

[1000]	1009
[1004]	
[1008]	
[1009]	10
[1010]	



Declaração de ponteiros

- Sintaxe:

<tipo> *<nome_da_variavel_ponteiro>;



Operador especial (*), indica que a variável armazena um endereço de memória, cujo conteúdo é do tipo especificado.

Qualquer tipo válido em C: pode-se utilizar os tipos padrão da linguagem C, como **char**, **int** e **float**, ou novos tipos definidos pelo programador.

- Exemplo:

```
float *f; //f é um ponteiro para variáveis do tipo float
int i, j, *p; //p é um ponteiro para variáveis do tipo inteiro
```

Tamanho de um ponteiro

- O tamanho de uma variável do tipo ponteiro é igual ao tamanho de um endereço de memória, **não importa o tipo.**
- Exemplo:

```
int *ip;  
char *cp;  
printf("size of int pointer: %lu\n", sizeof(ip));  
printf("size of char pointer: %lu\n", sizeof(cp));
```

```
size of int pointer: 8  
size of char pointer: 8
```

Por que ponteiros têm um tipo?

Tipos de Dados:

- Diferentes tipos de dados necessitam de um **número diferente de bytes** de memória para armazenar seu conteúdo.
- A declaração do tipo de dado indica o **número de bytes** reservados para conteúdos:

```
char a = 'A';           //1 byte  
int n = 1534;           //4 bytes  
float PI = 3.1415;      //4 bytes
```

Depende da arquitetura.
Para saber: `sizeof(tipo)`

Memória

'A'		1534				3.1415			...
1000	1001	1002	1003	1004	1005	1006	1007	1008	...

- O endereço de uma variável é sempre o **MENOR** dos endereços que ela ocupa, isto é, o **endereço inicial**.
- Um ponteiro para um tipo qualquer indica também o **número de bytes** que esse tipo ocupa.

Operadores de ponteiros

&<variável>

Operador unário que devolve o **endereço** de memória de seu operando.

Memória:

[endereço] conteúdo

```
#include <stdio.h>
int main(){
    int x = 1;
    int y = 10;

    printf("O end de x eh %p \n", &x);
    printf("O end de y eh %p \n", &y);

    return 0;
}
```

[1000]	1
[1004]	10
[1008]	
[1009]	
[1010]	
⋮	⋮

```
O end de x eh 1000
O end de y eh 1004
```

Operadores de ponteiros

- O operador unário **&** permite obter o endereço de uma variável comum, exemplo:

```
int aux;  
scanf("%d", &aux);
```

- Quando um ponteiro é declarado, pode-se inicializá-lo usando o operador **&**, fazendo o ponteiro receber um endereço de uma variável já existente.

Operadores de ponteiros

```
#include <stdio.h>

int main(){
    int x = 1; //variável int
    float y = 10.0; //variável float

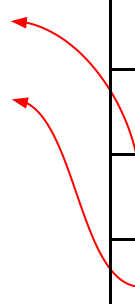
    int *p1; //ponteiro para variável int
    float *p2; //ponteiro para variável float

    p1 = &x; //p1 aponta para x
    p2 = &y; //p2 aponta para y

    printf("O end de x eh %p \n", p1);
    printf("O end de y eh %p \n", p2);

    return 0;
}
```

Endereço	Conteúdo
x:[1000]	1
y:[1004]	10
p1:[1008]	1000
p2:[1009]	1004
[1010]	
⋮	⋮



```
O end de x eh 1000
O end de y eh 1004
```

Operadores de ponteiros

***<ponteiro>**

Operador unário que devolve o **conteúdo** da variável localizada no endereço que o segue (ponteiro)

```
int main(){  
    int x = 31;  
    int y;  
  
    int *p1;  
  
    p1 = &x;  
  
    y = *p1;  
  
    printf("O valor de y eh %d", y);  
  
    return 0;  
}
```

Memória:

[endereço]	conteúdo	
x:[1000]	31	
y:[1004]	31	
p1:[1008]	1000	
[1009]		
[1010]		
⋮	⋮	

O valor de y eh 31

Atribuições de Ponteiros

- Como qualquer variável, um ponteiro **p1** pode ser usado no **lado direito** de um **comando de atribuição** para atribuir o seu valor para um outro ponteiro **p2**.
- Neste caso, o endereço armazenado em **p1** passa a ser conteúdo apontado por **p2** também.

```
#include <stdio.h>
int main(){
    int x = 200, *p1, *p2;
    p1 = &x; // p1 aponta para x
    p2 = p1; // p2 recebe p1 e também passa a apontar para x
    printf("x - %d, p1=%p e p2=%p\n", x, p1, p2); // endereço de x
    printf("x - %d, p1=%d e p2=%d\n", x, *p1, *p2); // conteúdo de x
    return 0;
}
```

```
x - 200, p1=0x7ffeaa46ab2c e p2=0x7ffeaa46ab2c
x - 200, p1=200 e p2=200
```

Uso dos operadores & e *

- Exemplo:

```
int x=1, y=2, z[2];  
int *ip;  
  
ip = &x; //ip agora aponta para x  
y = *ip; //y recebe o valor atual de x  
*ip = 0; //x tem agora valor igual a 0  
ip = &z[0]; //ip agora aponta para z[0]
```

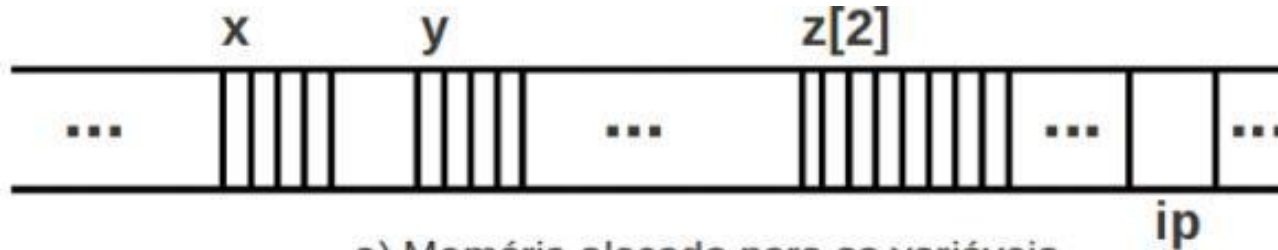
- A declaração **int *ip** diz que a expressão ***ip** é um **int**

Ex.: `*ip = *ip + 10;` //incrementa o inteiro `*ip` de 10

- Atenção:**

`ip = ip + 10;` //incrementa o endereço armazenado em `ip` de 10

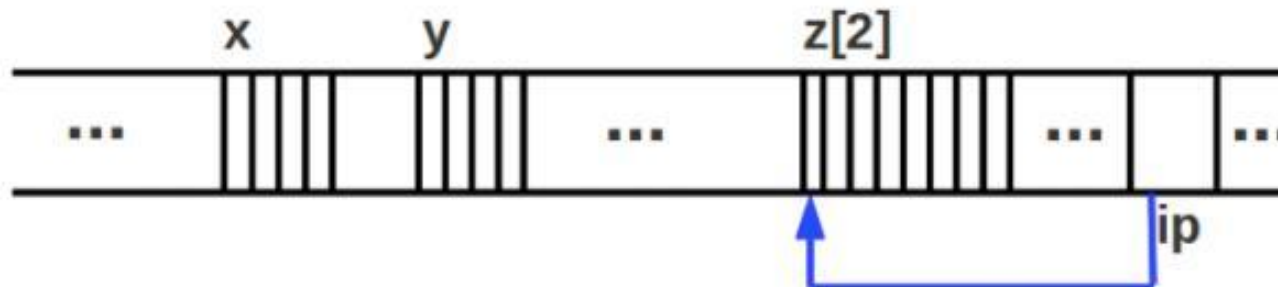
Uso dos operadores & e *



a) Memória alocada para as variáveis



b) `ip = &x;` (ip agora aponta para x)
`y = *ip;` (y recebe o valor atual de x)



c) `ip = &z[0];` (ip agora aponta para z[0])

Inicialização de ponteiros

- Ponteiros não devem ser utilizados sem **prévia inicialização**.

Exemplo típico:

```
int x;  
float pi = 3.14;  
  
int *ptr_x = &x;  
float *ptr_to_pi = &pi;
```

- No entanto, podem existir situações em que não se quer inicialmente que um ponteiro aponte uma variável.
- Nesse caso, deve-se inicializar o ponteiro com **NULL**. Para isso, deve-se incluir `<stdlib.h>`:

`int *ptr = NULL;`
- Em geral, isso é útil quando queremos testar se um dado ponteiro está ou não apontando para algum endereço.

Inicialização de ponteiros

- Manipular uma variável do tipo ponteiro sem inicializá-la com um endereço de memória **válido**.
- Inicializar uma variável ponteiro diretamente com um valor inteiro (pode ser um endereço inválido)

```
int *ip, i, j;  
*ip = 5; // incorreto: antes inicializar ip!  
ip = &i;  
*ip = 5; // correto!  
ip = 10; // incorreto: 10 pode ser  
//um endereço inválido para o programa!  
ip = &j; // correto!
```

Exercício

- Quais serão os **conteúdos** das variáveis ao fim do programa?

```
#include <stdio.h>
int main(){
    int a, *x, *y;
    a = 10;
    x = &a;
    y = x;
    printf("a = %d, *x = %d, *y = %d \n", a, *x, *y);
    printf("a = %d, x = %p, y = %p\n\n", a, x, y);
    *x = 20;
    printf("a = %d, *x = %d, *y = %d \n", a, *x, *y);
    printf("a = %d, x = %p, y = %p\n\n", a, x, y);

    return 0;
}
```

Variável	Endereço	Conteúdo
a	end_a	?
x	end_x	?
y	end_y	?

Exercício

- Quais serão os **conteúdos** das variáveis ao fim do programa?

```
#include <stdio.h>
int main(){
    int a, *x, *y;
    a = 10;
    x = &a;
    y = x;
    printf("a = %d, *x = %d, *y = %d \n", a, *x, *y);
    printf("a = %d, x = %p, y = %p\n\n", a, x, y);
    *x = 20;
    printf("a = %d, *x = %d, *y = %d \n", a, *x, *y);
    printf("a = %d, x = %p, y = %p\n\n", a, x, y);

    return 0;
}
```

Variável	Endereço	Conteúdo
a	end_a	20
x	end_x	end_a
y	end_y	end_a

Execução real:

```
a = 10, *x = 10, *y = 10
a = 10, x = 0x7ffdf5a512fc, y = 0x7ffdf5a512fc

a = 20, *x = 20, *y = 20
a = 20, x = 0x7ffdf5a512fc, y = 0x7ffdf5a512fc
```

Exercício

- O que será impresso na tela?

```
#include <stdio.h>

int main(){
    int x=5,*px;

    printf("1) x = %d *px = %d px = %p\n", x, *px, px);
    px = &x;

    printf("2) x = %d *px = %d px = %p\n", x, *px, px);
    *px = 10;

    printf("3) x = %d *px = %d px = %p\n", x, *px, px);

    return 0;
}
```

Exercício

- O que será impresso na tela?

```
#include <stdio.h>

int main(){
    int x=5,*px;

    printf("1) x = %d *px = %d px = %p\n", x, *px, px);
    px = &x;

    printf("2) x = %d *px = %d px = %p\n", x, *px, px);
    *px = 10;

    printf("3) x = %d *px = %d px = %p\n", x, *px, px);

    return 0;
}
```

O ponteiro px não foi inicializado neste ponto ainda

Não temos como saber qual é o conteúdo

Não temos como saber qual é o endereço

Execução real:

```
1) x = 5 *px = 1 px = 0x7ffc12eb7740
2) x = 5 *px = 5 px = 0x7ffc12eb7658
3) x = 10 *px = 10 px = 0x7ffc12eb7658
```

Erro comum

- Qual é o erro?

```
#include <stdio.h>

int main()
{
    float x;
    int *p;

    x = 100;
    p = &x;
    printf("x=%f p=%p\n\n", x, p);
    return 0;
}
```

Erro comum

- Qual é o erro?

```
#include <stdio.h>

int main()
{
    float x;
    int *p;

    x = 100;
    p = &x;
    printf("x=%f p=%p\n\n", x, p);
    return 0;
}
```

Erro de compilação!
p é um ponteiro para **int**,
logo não pode apontar para
uma variável do tipo **float**.

Erro comum

- Qual é o erro?

```
#include <stdio.h>

int main()
{
    int x;
    int *p;

    x = 100;
    *p = x;
    printf("x=%d p=%p\n\n", x, p);
    return 0;
}
```

Erro comum

- Qual é o erro?

```
#include <stdio.h>

int main()
{
    int x;
    int *p;

    x = 100;
    *p = x;
    printf("x=%d p=%p\n\n", x, p);
    return 0;
}
```

Provável erro de execução!

- **p** não foi inicializado quando foi criado.
- Logo, não sabemos para que endereço de memória ele aponta.
- A atribuição pode alterar o valor de alguma outra variável do programa.